



# ROB 2140

## Studio 5000 STRUCTURED TEXT LABS

John Sefcovic  
Oakland Community College



For each lab, the **Watch** must be **displayed** during program execution for credit.

You may use **Copy** and **Paste** to make a copy of the text in this document to paste into the PLC editor.

**DO NOT** copy the instructions structures that must be dragged into the editor. These commands are noted in a **red box**.

Drag those instructions from the **Instruction Palette** into the PLC editor.

```
instruction text;
instruction procedure;
instruction text;
```

### TABLE OF CONTENTS

#### Studio 5000 STRUCTURED TEXT LABS

|                                                   |    |
|---------------------------------------------------|----|
| LAB: SS1.....                                     | 1  |
| LAB: WASHER STATION TIMER .....                   | 5  |
| LAB: TEMPERATURE CONVERSION with INDICATORS ..... | 11 |
| EXAMPLE CODE .....                                | 22 |

© 2025 John Sefcovic and Oakland Community College. This work is intended for educational purposes. The author and publisher make no express or implied warranty of any kind, nor shall they be liable in any event for any type of damage or injury, in connection with or arising out of these materials. Studio 5000 Logix Designer, Studio 5000® Logix Emulate, FactoryTalk ME, and RSLinx are registered trademarks of Rockwell Automation, Inc. of Milwaukee, Wisconsin. All such names are used only in an editorial fashion to the benefit of the trademark owner without any intention of trademark infringements. This work may be reproduced and redistributed, in whole or in part, without alteration and without prior written permission, solely by educational institutions for nonprofit administrative or educational purposes provided all copies contain this copyright statement and the Oakland Community College logo. This work is reproduced and distributed with the permission of John Sefcovic. No other use is permitted without the express prior written permission of John Sefcovic or Oakland Community College.



**LAB: SS1**  
**PROCESS MODEL: SS1**

START - STOP circuit for a single motor control with ON - OFF indicators.

**INPUTS:**

| Address          | Controller Tag | Descriptions              |
|------------------|----------------|---------------------------|
| Local:3:I.Data.0 | STOP           | Pushbutton Momentary N.C. |
| Local:3:I.Data.1 | START          | Pushbutton Momentary N.O. |

**OUTPUTS:**

| Address          | Controller Tag | Descriptions             |
|------------------|----------------|--------------------------|
| Local:4:O.Data.0 | PL1_OFF        | Pilot Light 1 OFF (Red)  |
| Local:4:O.Data.1 | PL2_ON         | Pilot Light 1 ON (Green) |
| Local:4:O.Data.2 | MOTOR          | Motor Control            |

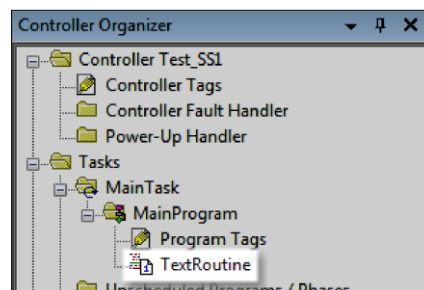
**PROGRAM LOCATION:**

Studio 5000 Folder  
Example Languages Folder  
Structured Text Folder

**FILE NAME:** Text\_SS1

**ROUTINE:**

Expand the Controller Organizer and select the routine **TextRoutine**.



*continue*

---

## Note on Program:

The [ := ] tells the controller to also clear the outlet tags whenever the controller enters the Run mode.

---

## TEXT ROUTINE

---

```
//START-STOP pushbuttons to turn On MOTOR
```

```
MOTOR := _____ AND ( _____ OR _____ );
```

Fill in 5 tags and a keyword

Using the motor to determine if Pilot light is on or off.

```
//Indicators status based on MOTOR On or Off
```

```
PL1_OFF:= _____; 2 things to enter in here
```

```
PL2_ON[ := ] _____;
```

---

## WATCH WINDOW

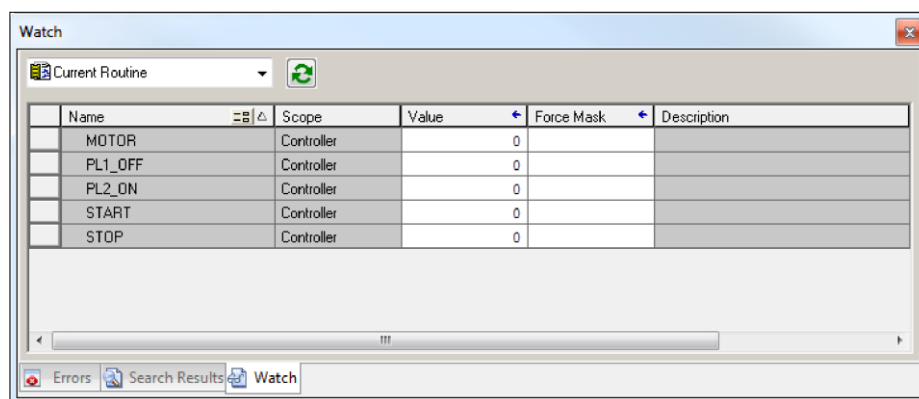
---

From the Menu Toolbar, select **View** and select **Watch**.

**Notice:** The tabs at the bottom of the Watch Window. The same dialog window is shared, with the verify **Errors** and **Search** results. Select the **Watch** tab to view tags.

By default, the tags in the current routine are displayed.

The tags are listed in this window, and a Quick Watch does not need to be created and saved.



---

## DOWNLOAD

---

Download the PLC program and place the PLC into RUN.

*An HMI has been created to allow you to test the logic, **SS1\_HMI**.*

The HMI is a runtime file.

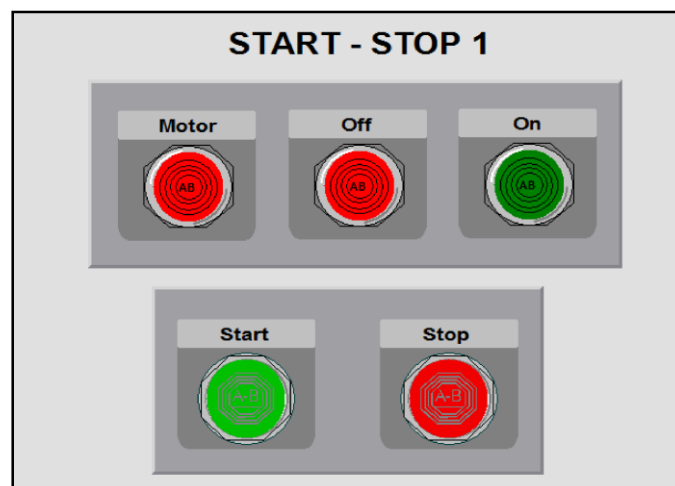
See desktop PDF document FactoryTalk View Studio Machine Edition Procedures on page 15 Appendix B: STATION RUNTIME APPLICATION.

*continue*

---

## TESTING

---



*Notice the tag **Value** STOP pushbutton is set to **1**.*

*Notice the tag **Value** PL1\_OFF indicator is set to **1**.*

1. On the HMI, left click and hold the **START** pushbutton.

**Value** field:

- The START pushbutton is set to **1**.
- The PL1\_OFF indicator will turn Off, reset to **0**.

*continue*

- MOTOR will turn On, set to **1**.
- The PL2\_ON will turn On, set to **1**.
- Release the START pushbutton.
- The **Value** changes to **0**.

2. Left click on the **STOP** pushbutton.

**Value** field:

- STOP changes to **0** *to emulate the pressing of the STOP pushbutton.*
- The PL1\_OFF indicator will turn On, set to **1**.
- MOTOR will turn Off, reset to **0**.
- The PL2\_ON will turn Off, reset to **0**.
- Release the STOP pushbutton.
- The **Value** changes to **0**.



## LAB: WASHER STATION TIMER

### PROCESS MODEL: WASH\_TON

Create a program that when a momentary normally open pushbutton is pressed and held, two clamp solenoids will be energized. A normally open limit switch for each clamp will be set when that clamp is in position. When the two clamps are in position, a spray wash will clean the part for ten seconds. Releasing the pushbutton will turn off the spray wash and clamps.

#### INPUTS:

| Address          | Controller Tag | Descriptions          |
|------------------|----------------|-----------------------|
| Local:3:I.Data.0 | PUSHBUTTON     | Pushbutton Wash Cycle |
| Local:3:I.Data.1 | SWITCH_CLAMP1  | Limit Switch Clamp 1  |
| Local:3:I.Data.2 | SWITCH_CLAMP2  | Limit Switch Clamp 2  |

#### OUTPUTS:

| Address          | Controller Tag  | Descriptions     |
|------------------|-----------------|------------------|
| Local:4:O.Data.0 | SOLENOID_CLAMP1 | Clamp 1 Solenoid |
| Local:4:O.Data.1 | SOLENOID_CLAMP2 | Clamp 2 Solenoid |
| Local:4:O.Data.2 | WASH_VALVE      | Spray Wash Valve |

#### TIMER:

| Param/Local Tag | Data Type | Descriptions  |
|-----------------|-----------|---------------|
| WASH_TIMER      | FBD_Timer | Process Timer |

#### PROGRAM LOCATION:

Studio 5000 Folder  
Example Languages Folder  
Function Block Folder

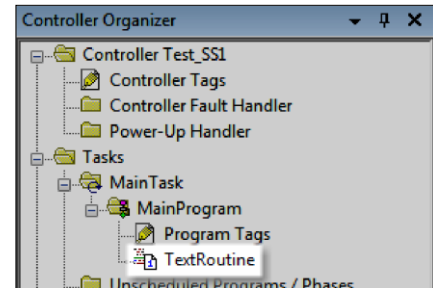
FILE NAME: Text\_WASH\_TON



---

## ROUTINE:

Expand the Controller Organizer and select the routine **TextRoutine**.



---

## Note on Program:

The [ := ] tells the controller to also clear the outlet tags whenever the controller enters the Run mode.

---

## TEXT ROUTINE

---

The timer is a Program Tag, and the preset and status bits may be used in the program.

**TONR** instruction, **Timer/Counter** tab

There are three versions, differences are on the controlling of the wash valve.  
*Only one version, your choice, is required for the lab.*

Version 1: Using the pushbutton, clamp switches, and timer done to control wash valve.

```
//START pushbuttons to turn On/Off CLAMP solenoids
If _____ then
SOLENOID_CLAMP1[:=] ____;
SOLENOID_CLAMP2[:=] ____;
else;
SOLENOID_CLAMP1:=____;
SOLENOID_CLAMP2:=____;
end_if;

//Timer structure for time to wash
WASH_TIMER.____ := 5000;
WASH_TIMER.TimerEnable := _____ & _____ & _____;
TONR ( _____ );
```



```
//Turn On/Off WASH valve when PUSHBUTTON and CLAMP 1 and CLAMP 2
//switches are closed, and the time is not complete
If _____ & _____ & _____ & _____ then
    _____[:=]_____;
else;
WASH_VALVE:=_____;
end_if;
```

### Version 2: Using the timer enable and done bits to control wash valve.

```
//START pushbuttons to turn On/Off CLAMP solenoids
If PUSHBUTTON then
SOLENOID_CLAMP1[:=]_____;
SOLENOID_CLAMP2[:=]_____;
else;
SOLENOID_CLAMP1:=_____;
SOLENOID_CLAMP2:=_____;
end_if;

//Timer structure for time to wash
WASH_TIMER._____ := 5000;
WASH_TIMER._____ := _____ & _____ & _____;
TONR(_____);
```

```
//Turn On/Off WASH valve when the timer is enabled and the time is not
//complete
If _____ & _____ then
WASH_VALVE[:=]_____;
else;
WASH_VALVE:=_____;
end_if;
```

### Version 3: Using the timer's timer timing bit to control wash valve.

```
//START pushbuttons to turn On/Off CLAMP solenoids
If PUSHBUTTON then
SOLENOID_CLAMP1[:=]_____;
SOLENOID_CLAMP2[:=]_____;
else;
SOLENOID_CLAMP1:=_____;
```





```

SOLENOID_CLAMP2:=____;
end_if;
//Timer structure for time to wash
WASH_TIMER.____ := 5000;
WASH_TIMER.____ := _____ & _____ & _____;
TONR ( _____ );

//Turn On/Off WASH valve when timer is elapsing
If _____ then
WASH_VALVE[:]= _____;
else;
WASH_VALVE:= _____;
end_if;

```

---

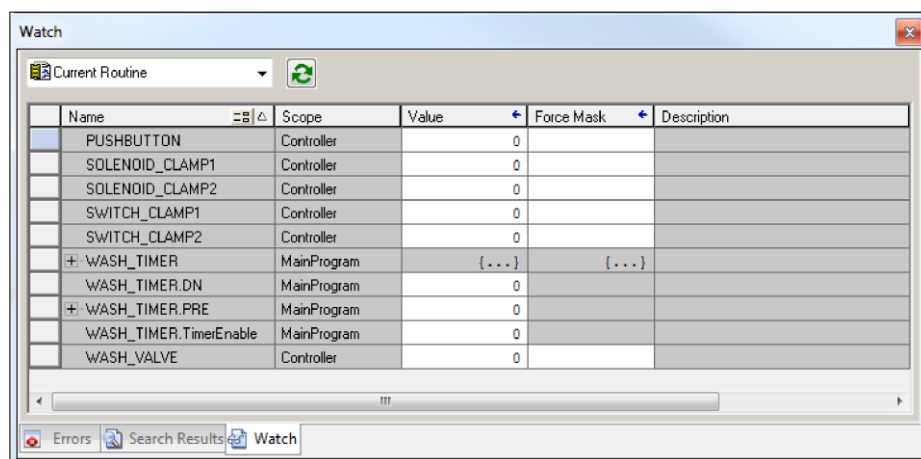
## WATCH WINDOW

---

From the Menu Toolbar, select **View** and select **Watch**.

**Notice:** The tabs at the bottom of the Watch Window. The same dialog window is shared, with the verify **Errors** and **Search** results. Select the **Watch** tab to view tags.

The tags are listed in this window and a Quick Watch does not need to be created and saved.



*Continue*



---

## DOWNLOAD

---

Download the PLC program and place the PLC into RUN.

*An HMI has been created to allow you to test the logic, **WASH\_TON\_HMI**.*

The HMI is a runtime file.

See desktop PDF document FactoryTalk View Studio Machine Edition Procedures on page 15 Appendix B: STATION RUNTIME APPLICATION.

---

## TESTING

---



1. On the HMI, left click and hold the **PUSHBUTTON**.

**Value** field:

- PUSHBUTTON changes to the value to **1**.
- The SOLENOID\_CLAMP1 and SOLENOID\_CLAMP2 will turn On, set to **1**.
- CLAMP\_SWITCH1 and CLAMP\_SWITCH2 close, set to **1**.
- The WASH\_VALVE turns On, set to **1**.
- The timer begins to elapse.

*continue*

- When the timer's preset and accumulated are equal, the Done bit is set.
- The Wash Valve turns Off, reset to **0**.

## 2. Release the **PUSHBUTTON**.

### **Value** field:

- Changes PUSHBUTTON value to **0**.
- Turns Off the SOLENOIDS\_CLAMP1 and SOLENOIDS\_CLAMP1, all reset **0**.
- Opens the SWITCH\_CLAMP1 and SWITCH\_CLAMP2, all reset **0**.
- The WASH\_VALVE turns Off, reset to **0**.
- Timer is reset.



## **LAB: TEMPERATURE CONVERSION with INDICATORS**

### **PROCESS MODEL: Text\_CONVERT**

#### **Main Routine**

The Routine **InputControlRoutine** for the conversion is called unconditionally.  
The Routine **IndicatorsRoutine** for the indicators is called unconditionally.

#### **ConvertBCD Routine**

A Ladder Logic routine if the Thumbwheel Switches are Enabled to convert the BCD value to a decimal Integer.

#### **InputControl Routine**

Create a program that will convert a Celsius temperature to a Fahrenheit temperature value. The maximum Celsius value will be 100 degrees.

The temperature to be converted will be entered via a the HMI as an integer value or the thumbwheel switch as a BCD value.

When the operator presses the CONVERT TEMPERATURE pushbutton, the program will perform the following error checks.

If the value entered has not changed, a duplicate error indicator will be lit. A check for the input temperature is 100 degrees or less will be performed. If it is above 100 degrees, turn on Over Value indicator will be lit. Any error will prevent a conversion of the TWS input value. The operator must then press the RESET pushbutton and enter the correct value.

#### **Convert Routine**

Converts the passed Celsius temperature from the Input Control Routine to a Fahrenheit value to be returned back to Input Control Routine.

#### **Indicators Routine**

If a conversion is valid, indicators will show the converted temperature is within a LOW LOW, LOW, MID, HIGH, or HIGH HIGH range.

There is no **Limit (LIM)** instruction in the Structured Text language, use the **Select - Case** construct.

*continue*



|         |           |            |
|---------|-----------|------------|
| Ranges: | Low Low   | 32 to 49   |
|         | Low       | 50 to 99   |
|         | Mid       | 100 to 149 |
|         | High      | 150 to 199 |
|         | High High | 200 to 212 |

### I/O CONFIGURATION:

Slot 2 - Studio Emulate 5000 Controller

Slot 3 - 1756-IB16

Slot 4 - 1756-OB16E

Slot 5 - 1756-IB16

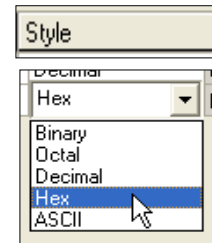
**NOTE:** All Inputs and Outputs are BOOLEAN Data Type shown in a DECIMAL Style, except TWS\_INPUT is a Double Integer Data Type shown in a HEX Style.

### INPUTS:

| Address           | Controller Tag | Descriptions                         |
|-------------------|----------------|--------------------------------------|
| Local:5:I.Data    | TWS_INPUT      | Thumbwheel Switch                    |
| Local:3:I.Data.14 | CONVERT_PB     | Convert Temperature N. O. pushbutton |
| Local:3:I.Data.15 | RESET_PB       | Data Error Reset, N. O. pushbutton   |

TWS\_INPUT will be in Hexidecimal. In the Style column in the TWS\_INPUT row .

Left click and select **Hex** from the drop-down list.



### OUTPUTS:

| Address          | Controller Tag | Descriptions           |
|------------------|----------------|------------------------|
| Local:4:O.Data.0 | ERROR_DUP_VAL  | Error Indicator        |
| Local:4:O.Data.1 | ERROR_OVER_VAL | Data Error, Over Value |
| Local:4:O.Data.2 | LL_INDC        | Low Low Indicator      |
| Local:4:O.Data.3 | L_INDC         | Low Indicator          |
| Local:4:O.Data.4 | M_INDC         | Mid Indicator          |
| Local:4:O.Data.5 | H_INDC         | High Indicator         |
| Local:4:O.Data.6 | HH_INDC        | High High Indicator    |



## INTERNALS:

| Param/Local Tag    | Data Type   | Descriptions                     |
|--------------------|-------------|----------------------------------|
| ONE_SHOT_CONTROL   | FBD_ONESHOT | One Shot                         |
| C_TEMP             | DINT        | Celsius Temperature              |
| C_TEMP_STORAGE     | DINT        | Storage of Celsius Temperature   |
| C_TEMP_CONVERT     | DINT        | Celsius Temperature Parameter    |
| F_TEMP             | DINT        | Fahrenheit Value                 |
| F_TEMP_CONVERT     | DINT        | Fahrenheit Temperature Parameter |
| HMI_CONVERT_PB     | Boolean     | HMI Convert Button               |
| HMI_RESET_PB       | Boolean     | HMI Reset Button                 |
| HMI_ENABLE_DISABLE | Boolean     | HMI Enable/Disable Button        |

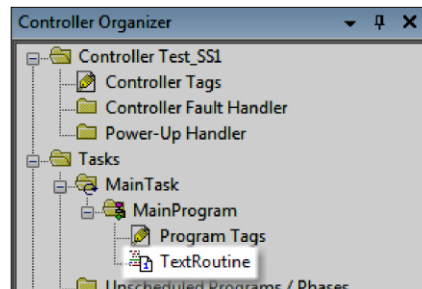
## PROGRAM LOCATION:

**Studio 5000 Folder**

**Example Languages Folder**

**Structured Text Folder**

**FILE NAME:** Text\_CONVERT



*continue*



---

## ROUTINES:

Expand the Controller Organizer and select the routine **TextRoutine**.

Add the routines:

**ConvertBCDRoutine**, Language: Ladder Logic.  
**InputControlRoutine**, Language: Structured Text.  
**ConvertRoutine**, Language: Structured Text.  
**IndicatorsRoutine**, Language: Structured Text.

---

## NOTES ON ROUTINES

---

### Name of Routine: TextRoutine

An unconditional Jump to Subroutine to **InputControlRoutine**.  
An unconditional Jump to Subroutine to **IndicatorsRoutine**.

---

### Name of Routine: InputControlRoutine

The one-shot is controlled by the CONVERT PUSHUTTON.

If the Thumb Wheel Switches are enabled, by a zero value from the HMI, the one-shot call the ladder logic routine to convert the BCD input to an Integer value. **HMI\_ENABLE\_DISABLE**

The routine **ConvertBCDRoutine** is called for one scan only for the conversion of the TWS BCD value

Controlled by the one-shot, the alarms are set if an error condition exist. The setting of the alarm status bits, and any data in Structured Text is retentive (latched) by default.

The RESET PUSHBUTTON resets (unlatches) the alarm status bits.

A set one-shot and no errors causes:

the Celsius Temperature to be stored.

Jump to Subroutine to **ConvertRoutine** to pass the Input Parameter Celsius temperature value and Return Parameter the Fahrenheit temperature value

**JSR( name of routine,1, input parameter , return parameter)**

The 1 in the Jump to Subroutine indicates the number of Input Parameters listed.



---

### Name of Routine: ConvertRoutine

A Subroutine (SBR) instruction for the Celsius temperature to be converted tag name.  
The equation to convert a Celsius temperature value to a Fahrenheit temperature value.  
A Return (RET) instruction with the tag name of the converted Fahrenheit temperature.

*SBR(input parameter)* the input parameter is the tag name used in the routine.

Equation:  $F = (C * 9) / 5 + 32$

*RET(return parameter)* the return parameter is the tag name used in the routine.

---

### Name of Routine: IndicatorsRoutine

The Select Case Of construct sets the appropriate indicator for the Low Low, Low, Mid, High, or High High range. In the Case Of construct...

- a span of values can be selected by `value .. value`
- the status, 0 or 1, for each range must be defined as Off (0) or On (1). Remember the assignment to a Tag in Structured Text is retentive (latched) and when a different range is selected, the status of the previous range must be reset.

---

## TEXT ROUTINES

---

### Instruction Palette

**JSR** instruction, **Program Control** tab.  
**SBR** instruction, **Program Control** tab.  
**RET** instruction, **Program Control** tab.  
**OSRI** instruction, **Bit** tab.

---

*continue*





---

## Text Routine

```
JSR ( _____ ) ;  
JSR ( _____ ) ;
```

Need to drag it in.

---

## ConvertBCDRoutine

There is no **Convert to Integer (FRD)** instruction in the Structured Text language, a Jump to Subroutine (JSR) instruction to call a ladder Logic routine with a FRD to convert the Binary Coded Decimal BCD to an Integer value.



---

## InputControlRoutine

```
//One shot for CONVERT PUSHBUTTON or HMI CONVERT PUSHBUTTON.  
ONE_SHOT_CONTROL. _____ := _____ OR _____ ;
```

```
OSRI ( _____ ) ;
```

```
//Subroutine call if Thumb Wheel Switches are enabled.  
If _____ & ONE_SHOT_CONTROL. _____ then  
JSR ( _____ ) ;  
end_if;
```

```
/*If no previous Errors, check for Duplicate or Over Values  
and set alarms.*/  
If ONE_SHOT_CONTROL. _____ & _____ ERROR_DUP_VALUE & _____  
ERROR_OVER_VAL then  
ERROR_DUP_VALUE := _____ = _____ ;  
ERROR_OVER_VAL := _____ _____ ;  
end_if;
```

```

//Error RESET PUSHBUTTON or HMI RESET PUSHBUTTON and clearing alarms.
If _____ OR then _____
ERROR_DUP_VAL := ____;
ERROR_OVER_VAL := ____;
end_if;

/*If no errors, store the Celsius value and call the conversion
routine with the Celsius temperature to return the Fahrenheit
temperature.*/
If ONE_SHOT_CONTROL. _____ & _____ & _____
then
_____ := _____;
JSR(_____, 1, _____, _____);
end_if;

```

---

### **ConvertRoutine**

```

//Convert a Celsius temperature to a Fahrenheit temperature.
SBR(_____) ;
F_TEMP_CONVERT := (_____ * ____ ) / ____ + ____;
RET(_____) ;

```

### **IndicatorsRoutine**

```

//Select Case to set range indicator.
Case F_TEMP of
32..49:    LL_INDC := ____;
           L_INDC  := ____;
           M_INDC  := ____;
           H_INDC  := ____;
           HH_INDC := ____;
50..99:    LL_INDC := ____;
           L_INDC  := ____;
           M_INDC  := ____;
           H_INDC  := ____;
           HH_INDC := ____;

```



```

100..149: _____ := _____;
           _____ := _____;
           _____ := _____;
           _____ := _____;
           _____ := _____;

____. __. ____: _____ := _____;
               _____ := _____;
               _____ := _____;
               _____ := _____;
               _____ := _____;

____. __. ____: _____ := _____;
               _____ := _____;
               _____ := _____;
               _____ := _____;
               _____ := _____;

end_case;

```

## WATCH WINDOW

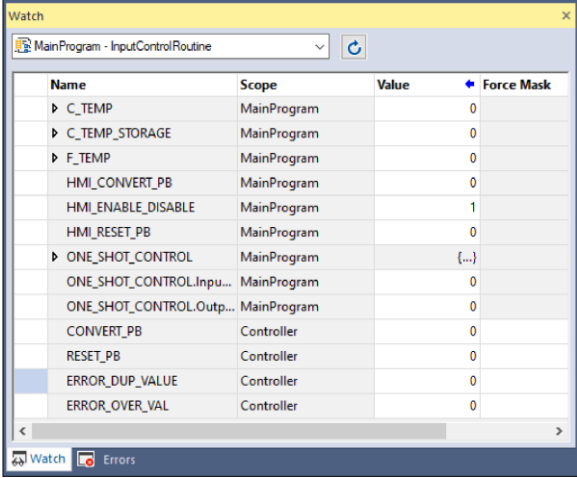
### OPEN WATCH WINDOW

From the Menu Toolbar, select **View** and select **Watch**.

**Note:** When a routine is opened in the Editor all the Tags in the routine will be displayed.

From the drop down select the name of the routine to display in the Watch Window.

*You will be asked to display various routines in the Watch Window during sign-off.*



| Name                     | Scope       | Value | Force Mask |
|--------------------------|-------------|-------|------------|
| ▸ C_TEMP                 | MainProgram | 0     |            |
| ▸ C_TEMP_STORAGE         | MainProgram | 0     |            |
| ▸ F_TEMP                 | MainProgram | 0     |            |
| HMI_CONVERT_PB           | MainProgram | 0     |            |
| HMI_ENABLE_DISABLE       | MainProgram | 1     |            |
| HMI_RESET_PB             | MainProgram | 0     |            |
| ▸ ONE_SHOT_CONTROL       | MainProgram | {...} |            |
| ONE_SHOT_CONTROL.Inpu... | MainProgram | 0     |            |
| ONE_SHOT_CONTROL.Outp... | MainProgram | 0     |            |
| CONVERT_PB               | Controller  | 0     |            |
| RESET_PB                 | Controller  | 0     |            |
| ERROR_DUP_VALUE          | Controller  | 0     |            |
| ERROR_OVER_VAL           | Controller  | 0     |            |

---

## DOWNLOAD

---

Download the PLC program and place the PLC into RUN.

Testing is done in the same TEMPERATURE CONVERSION with INDICATORS HMI you created for the Ladder Logic program.

Open **FactoryTalk View ME Station**.

See [FactoryTalk View Studio Machine Edition Procedures](#)

---

## TESTING

---

---

### ENABLE HMI

1. From the MAIN screen select the **MAINTENANCE** button.



2. In the MAINTANCE screen select **TWS DISABLED**.



3. Select the go to **INPUT** button.

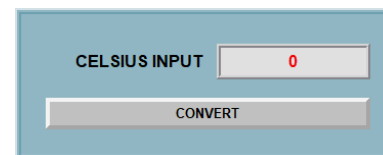


---

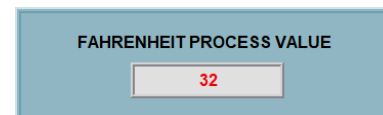
### TESTING CELSIUS TO FAHRENHEIT CONVERSION

1. Set a 0 value for the **CELSIUS INPUT**.

2. Press the **CONVERT** pushbutton.



3. Check for the correct converted temperature, **should be 32° F**.



4. Press the **RANGE** button.

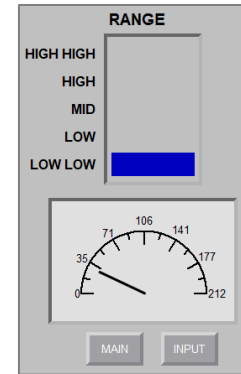


*continue*

4. Check for the correct **Range Indicator and Gauge** value.

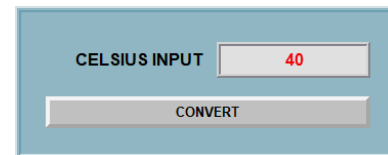
Repeat the steps 1 through 7 with the other Celsius values into **C\_TEMP** to test conversion and the Range Indicators.

| Celsius | Fahrenheit | Range     |
|---------|------------|-----------|
| 0°      | 32°        | Low Low   |
| 30°     | 86°        | Low       |
| 60°     | 140°       | Mid       |
| 80°     | 176°       | High      |
| 100°    | 212°       | High High |

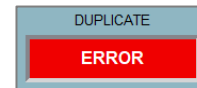


## TESTING DUPLICATE ERROR

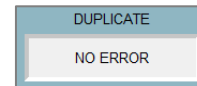
1. Enter a Celsius value of 40 into **CELSIUS INPUT**.
2. Press the **CONVERT** pushbutton.



3. Again, press the **CONVERT** pushbutton.  
DUPLICATE ERROR indicator should be ON.

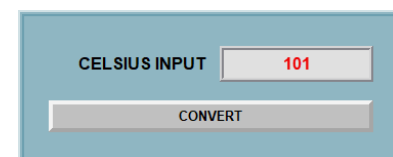


4. Press the **RESET** pushbutton.  
DUPLICATE ERROR indicator should be OFF.



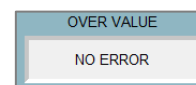
## TESTING OVER TEMPERATURE VALUE ERROR WITHOUT HMI

1. Enter a Celsius value of 101 into **CELSIUS INPUT**.
2. Press the **CONVERT** pushbutton.



OVER VALUE ERROR indicator should be ON.

3. Toggle (ON-OFF) the **RESET\_PB** pushbutton.  
OVER VALUE ERROR indicator should be OFF.



*continue*



## YOU WILL BE REQUIRED TO DEMONSTRATE THE THUMBWHEEL SWITCHES WHEN THE HMI IS GRADED.

---

### ENABLE HMI

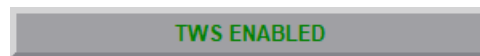
1. Select the go to **MAIN** button.



2. Select the **MAINTENANCE** button.



3. Change to **TWS ENABLED**.



- Select the go to **INPUT** button.



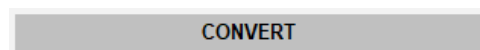
---

### TO ENTER BCD THUMBWHEEL VALUE ONLINE IN STUDIO 5000

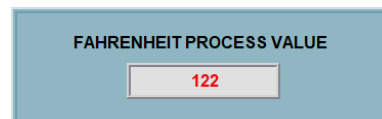
1. Open the **Controller Tags** table.
2. Select **Monitor Tags** tab.
3. Select the **Value** column field for the **TWS\_INPUT** tag.

| Name        | Value        | Force Mask | Style |
|-------------|--------------|------------|-------|
| ▶ TWS_INPUT | 16#0000_0050 |            | Hex   |

4. Enter a Celsius value 50 and press **Enter**.
5. Press the HMI **CONVERT** pushbutton.



6. Check for the correct converted temperature, [should be 122° F](#).



## EXAMPLE CODE

### Momentary Pushbutton Normally Open

INDICATOR On when pushbutton pressed, that is, a logic 1

```
INDICATOR := PUSHBUTTON;
```

### Momentary Pushbutton Normally Closed

INDICATOR On when pushbutton pressed, that is, a logic 0.

```
INDICATOR := NOT PUSHBUTTON;
```

### Maintained Switch ON/OFF

```
If SWITCH then;  
INDICATOR := 1;  
else;  
INDICATOR := 0;  
end_if;
```

### Holding Circuit

START - Normally Open

STOP - Normally Closed

```
MOTOR := STOP AND (START OR MOTOR);
```

### Timer On

```
DWELL.PRE := 2000;  
DWELL.EnableIn := SWITCH  
TONR(DWELL);  
  
INDICATOR_ENABLED := DWELL.EN  
INDICATOR_TIMING := DWELL.TT  
INDICATOR_DONE := DWELL.DN
```



## Move

```
VALUE2 := VALUE1;
```

## One Shot Rising

```
ONE_SHOT.InputBit := PUSHBUTTON;  
OSRI(ONE_SHOT);  
STATE := ONE_SHOT.OutBit;
```

```
If ONE_SHOT.OutBit then  
VALUE2 := VALUE1;  
end_if;
```

## Select Case

```
Case SENSOR of  
10..20:  L_VALUE := 1;  
          M_VALUE := 0;  
          H_VALUE := 0;  
21..30:  L_VALUE := 0;  
          M_VALUE := 1;  
          H_VALUE := 0;  
31..40:  L_VALUE := 0;  
          M_VALUE := 0;  
          H_VALUE := 1;  
end_case;
```

